

CO1 Assessment Task 1: Case Study Analysis
Software Engineering

Name: Mohammed Fareeth S A (1/251)
Institution: SIMATS School of Engineering

Case Study 1: Telemedicine Mobile App Platform

1. Understanding the Problem Statement

A healthcare company is building a telemedicine mobile app so patients can consult doctors remotely. The underlying problem has two layers: first, an actual product needs to be built (patient-doctor remote consultation), and second, the development process itself is unstable because the Product Owner keeps changing the backlog mid-sprint, which slows the team down and confuses stakeholders about what is actually being delivered.

Objectives: deliver a secure, reliable telemedicine platform that lets patients book and attend remote consultations, while fixing the requirement-churn problem so the team can deliver predictably sprint after sprint.

Scope: covers the patient-facing app, a doctor-facing portal, appointment scheduling, video consultations, e-prescriptions, and the Agile process changes needed to stabilize how requirements flow into the team.

2. Key Stakeholders and Challenges

Patients: end users who need a simple, trustworthy way to consult a doctor without visiting a clinic.

Doctors: provide consultations; need a reliable, low-friction interface that doesn't add administrative burden.

Product Owner: owns and prioritizes the backlog; responsible for keeping requirements stable once a sprint begins.

Scrum Master / development team: build the product and are directly affected by mid-sprint scope changes.

Hospital administration: business stakeholders who want the product to reflect operational needs, but whose late requests are the source of backlog churn.

Regulatory bodies: govern how patient health data must be handled and stored.

The major challenge is that stakeholders (hospital administration in particular) surface new requirements informally and late, and without a disciplined backlog-refinement process, those requests get pulled directly into an active sprint - creating confusion for the team and unpredictable delivery for everyone else.

3. Analysis of the Current Approach

The team already uses Scrum, which is the right starting point, but it is being run without the supporting discipline that makes Scrum work.

Strengths: iterative delivery model, and a team that is at least structured around sprints and ceremonies.

Limitations: no formal backlog refinement before sprint planning, no 'Definition of Ready' for backlog items, and no agreed prioritization method - so whichever stakeholder speaks to the PO last effectively decides what gets built next.

Gaps: missing communication cadence between the PO and hospital administration, no change-control process for urgent requests, and no metric tracking how often the backlog changes mid-sprint.

4. Proposed Software Solution

Beyond fixing the process, the platform itself needs a clear modular design:

- Patient Registration & Profile Module - account creation, medical history, document upload.
- Appointment Scheduling Module - doctor availability, booking, reminders.

- Video Consultation Module - real-time audio/video (e.g. WebRTC-based).
- e-Prescription Module - doctors issue digital prescriptions post-consultation.
- Payment & Billing Module - consultation fee processing.
- Admin Dashboard - hospital administration oversight and reporting.

Functional requirements: user registration/login, appointment booking, live video consultation, prescription generation, secure messaging, and notifications.

Non-functional requirements: data privacy compliance for health records, high availability (a dropped video call mid-consultation is unacceptable), low-latency video performance, and usability for patients who may not be tech-savvy.

Software engineering principles applied: modularity (each module built and deployed independently, e.g. video service separate from billing); scalability (cloud infrastructure that scales with concurrent video sessions); maintainability (clean API contracts between modules so hospital-specific customizations don't ripple through the whole system); reliability (redundant video servers with failover); usability (simple, accessible UI); and security (end-to-end encryption, role-based access control, and audit logging of who accessed what patient data).

5. Justification of Chosen Methodology/Framework

Scrum remains the right framework for this project, but strengthened rather than replaced. A Waterfall approach would be a poor fit here because healthcare requirements (regulatory rules, doctor workflow preferences, patient feedback) genuinely evolve as the product is used, and Waterfall locks requirements too early to accommodate that.

Scrum is justified because it supports incremental delivery and regular stakeholder feedback, which matters in healthcare software where usability issues surface only once real doctors and patients use the tool. The fix is not to abandon Scrum but to add the missing discipline: backlog refinement sessions before each sprint, a Definition of Ready, and a prioritization technique such as MoSCoW or Weighted-Shortest-Job-First so that the Product Owner has a structured way to rank incoming requests instead of reacting to whichever stakeholder spoke most recently.

6. Expected Outcomes and Limitations

Expected benefits: predictable sprint delivery, restored trust between the development team and stakeholders, and a secure, usable telemedicine platform that genuinely improves access to healthcare.

How it addresses the challenges: a disciplined backlog process removes the root cause of mid-sprint churn, while the modular architecture keeps the platform maintainable as new regulatory or clinical requirements arrive.

Risks and limitations: regulatory compliance is an ongoing burden, not a one-time task; rural patients may face poor connectivity affecting video quality; and hospital administration may initially resist the discipline of a locked sprint backlog.

Future enhancements: AI-assisted symptom triage before a consultation, wearable-device integration for vitals monitoring, and multi-language support for a wider patient base.

Case Study 2: Real-Time Smart Parking System

1. Understanding the Problem Statement

A startup wants to build a real-time smart parking system using sensors and a mobile booking app, but needs to confirm the idea is practical before committing investment. The problem is really two-fold: drivers currently waste time searching for parking, and the startup needs a structured way to evaluate feasibility before building anything.

Objectives: reduce the time drivers spend searching for parking, improve utilization of existing parking bays, and create a new revenue stream through app-based reservations.

Scope: sensor-equipped parking lots, a mobile app for drivers to view and reserve bays, and the backend systems connecting the two - starting with a pilot lot before any wider rollout.

2. Key Stakeholders and Challenges

Drivers: end users who want fast, reliable parking availability information.

Parking lot owners/operators: provide the physical space and benefit from better utilization and additional revenue.

The startup (business owner/investors): need proof the system is technically and financially viable.

Facility maintenance staff: responsible for keeping sensors calibrated and functioning.

Municipal authorities: relevant for public lots, where local parking regulations apply.

The main challenges are sensor reliability in outdoor conditions, uncertain up-front investment cost, integration with a lot's existing barriers or payment systems, and getting drivers to actually trust and adopt the app instead of falling back on old habits.

3. Analysis of the Current Approach

Existing process: most lots run on a first-come-first-served basis with static signage and no real-time visibility of free bays; payment, where required, is often manual or cash-based.

Strengths: simple to operate, no technology dependency, low ongoing IT cost.

Limitations: drivers waste time circling for a spot, lots are often under-utilized because availability isn't visible, and operators have no usage data to inform pricing or capacity planning.

Gaps: no real-time occupancy tracking and no way for a driver to reserve a spot in advance.

4. Proposed Software Solution

- Sensor/IoT Data Collection Module - ultrasonic or infrared sensors per bay reporting occupancy status.
- Real-Time Occupancy Engine - aggregates sensor data and maintains live availability state.
- Mobile Booking App - drivers view free bays, reserve, and navigate to them.
- Payment Gateway - handles reservation and parking fee payments.
- Admin/Analytics Dashboard - operators track utilization and revenue.

Functional requirements: live occupancy display, reservation, in-app navigation, payment processing, and operator-facing reporting.

Non-functional requirements: scalability to support multiple lots, reliability of sensor data even with intermittent connectivity, secure handling of payment data, and a simple enough interface that any driver can use it without a learning curve.

Software engineering principles applied: modularity (the sensor layer is decoupled from the app through a clear API, so sensor hardware can change without touching the app); scalability (cloud backend that scales per city or lot); reliability (redundant gateways and fallback logic when a sensor goes offline); security (encrypted payment transactions and secure IoT communication); and usability (a booking flow simple enough to complete in seconds).

5. Justification of Chosen Methodology/Framework

An Agile/Scrum approach with an initial Design Thinking phase is the right fit here, rather than Waterfall. Because this is a startup validating an unproven product, requirements around pricing models, reservation rules, and even sensor placement are likely to change once real usage data comes in from a pilot lot.

Agile supports building a minimal pilot first (a single lot), gathering feedback from drivers and the lot operator, and iterating before expanding to more locations - which matches the feasibility-first nature of this problem far better than committing to a fixed, upfront design.

6. Expected Outcomes and Limitations

Expected benefits: reduced search time for drivers, higher bay utilization and revenue for the operator, and usage data that can inform future pricing and expansion decisions.

How it addresses the challenges: real-time visibility solves the core driver pain point, while the pilot-then-scale approach directly answers the startup's feasibility question before large capital is committed.

Risks and limitations: sensor hardware failure or drift over time, poor connectivity in underground or enclosed lots, and the need for cooperation from lot owners to integrate with existing barrier systems.

Future enhancements: dynamic pricing based on demand, EV charging bay integration, and expansion across multiple lots in a city-wide network.

Case Study 3: Real-Time Tutoring Platform

1. Understanding the Problem Statement

A team is building a real-time tutoring platform and wants to reduce wasted effort using Lean thinking while improving code quality using Extreme Programming (XP). The problem is about building a live, reliable tutoring experience without the team drowning in rework, waiting, or unreleased half-built features.

Objectives: deliver a dependable live-tutoring product quickly, with minimal wasted development effort and high code quality suited to a real-time system.

Scope: covers live video/audio sessions, screen sharing, scheduling, and payments between students and tutors.

2. Key Stakeholders and Challenges

Students: end users who need a lag-free, easy-to-join learning experience.

Tutors: deliver lessons and need reliable session tools (screen share, whiteboard).

Product owner/company: prioritizes features and balances speed of delivery against quality.

Development team: responsible for applying Lean and XP practices day to day.

The central challenge is a real-time technical constraint (video/audio must stay in sync with minimal lag) combined with a process constraint (the team must move fast without accumulating waste such as half-finished features or late-discovered defects).

3. Analysis of the Current Approach

Existing process: development has been happening without a formalized waste-reduction or quality discipline, leading to several identifiable wastes - overproduction of features nobody asked for yet, developers waiting on reviews or on other teams' APIs, defects surfacing only during live sessions, unnecessary documentation overhead, frequent context-switching, and partially built features sitting unreleased.

Strengths: the team is motivated to improve and is open to adopting Lean and XP practices.

Gaps: there is no unified workflow that combines fast, waste-free delivery with the engineering discipline needed to keep defect rates low in a real-time product.

4. Proposed Software Solution

- User Management Module - student and tutor accounts, profiles.
- Live Session Module - real-time video/audio (e.g. WebRTC) and screen sharing.
- Scheduling & Booking Module - lesson slots and reminders.
- Payment & Billing Module - session fee handling.
- Analytics/Reporting Module - session history and usage patterns.

Functional requirements: account login, scheduling, live audio/video with screen share, session recording, and payment processing.

Non-functional requirements: low-latency performance for real-time interaction, reliability during reconnects, scalability for many concurrent sessions, and data protection given that some students may be minors.

Software engineering principles applied: modularity (the video/session service is separate from scheduling and payment, so each can evolve independently); scalability (video servers auto-scale with concurrent session load); maintainability (continuous refactoring keeps the codebase clean as features are added); reliability (automated testing and CI catch defects before they reach a live session); usability (simple join flow for students of varying ages); and security (encrypted sessions and careful handling of data belonging to minors).

5. Justification of Chosen Methodology/Framework

Lean combined with Extreme Programming (XP) is the appropriate framework here, rather than a heavier process like SAFe or a linear model like Waterfall. Lean's focus on eliminating waste (pulling work in small batches, visualizing work-in-progress, avoiding overproduction) keeps the team from over-building unnecessary features.

XP's engineering practices - pair programming, Test-Driven Development, Continuous Integration, small releases, and constant refactoring - directly address the quality risk in a real-time product, where a bug like audio drift can only be caught through disciplined testing rather than after the fact. Together, the two approaches match this project's dual need for speed and technical excellence far better than either one alone.

6. Expected Outcomes and Limitations

Expected benefits: faster, smaller releases; fewer defects reaching live sessions; and better use of the team's time and effort.

How it addresses the challenges: Lean's waste elimination directly reduces the wasted work identified earlier, while XP's testing and pairing discipline reduces the defect rate that real-time delivery cannot tolerate.

Risks and limitations: pair programming can feel like it slows the team down in the short term, and both Lean and XP depend heavily on team discipline and culture, which can be hard to maintain as the team grows larger.

Future enhancements: AI-based tutor-student matching, and adaptive analytics that highlight where a student is struggling during a session.

Case Study 4: Digital Banking Platform (Scaled Agile)

1. Understanding the Problem Statement

A large organization is building a digital banking platform that requires coordinating 10+ teams working on payments, credit scoring, fraud detection, and dashboards. The problem is fundamentally one of scale: individual teams may work well on their own, but delivering one coherent banking product across so many interdependent teams needs coordination that plain team-level Scrum cannot provide.

Objectives: deliver an integrated, secure digital banking platform where payments, credit scoring, fraud detection, and reporting all work together reliably, with well-synchronized delivery across all contributing teams.

Scope: covers the full banking platform - transaction processing, credit decisioning, real-time fraud detection, and customer/staff dashboards - along with the enterprise-scale coordination needed to build it.

2. Key Stakeholders and Challenges

Bank customers: end users of the platform's payments and account features.

Payments, credit scoring, fraud detection, and dashboard teams: each own a domain but must integrate tightly with the others.

Release Train Engineer, Product Management, System Architects, Business Owners: coordinate priorities, technical direction, and ROI across all the teams.

Regulatory and IT security bodies: set compliance and security requirements that all teams must satisfy.

The major challenges are cross-team dependencies (e.g. fraud detection needing a payments API), evolving regulatory requirements mid-development, and maintaining consistent security across shared components that many teams touch.

3. Analysis of the Current Approach

Existing process: teams likely operate with independent backlogs and their own sprint cadences, without a shared planning rhythm across the wider program.

Strengths: each team has strong domain expertise in its own area (payments, fraud, credit, dashboards).

Limitations: without a unified cadence, cross-team dependencies are often discovered late, causing integration delays and inconsistent risk management across teams.

Gaps: no synchronized planning event across all the teams, and no shared architectural governance to keep independently built services compatible.

4. Proposed Software Solution

- Payments Engine - transaction processing and settlement.
- Credit Scoring Engine - applicant risk evaluation.

- Fraud Detection Engine - real-time transaction monitoring and flagging.
- Customer Dashboard - account and transaction views for customers and staff.
- Identity & Access Management - authentication and authorization across all services.

Functional requirements: process transactions, score credit applications, flag fraudulent activity in real time, and present account information clearly.

Non-functional requirements: very high security given financial data (encryption, regulatory compliance such as PCI-DSS), high availability, scalability for large transaction volumes, and consistent performance for real-time fraud scoring.

Software engineering principles applied: modularity (each domain - payments, credit, fraud, dashboards - is built as an independent service with clear API boundaries); scalability (cloud-native services that scale independently under load); reliability (redundancy and disaster-recovery planning for critical financial services); maintainability (shared architectural standards enforced by System Architects so services stay compatible); and security (defense-in-depth, encryption, and strict access control across every shared component).

5. Justification of Chosen Methodology/Framework

The Scaled Agile Framework (SAFe) is the right fit here rather than plain Scrum, because Scrum is designed for a single team, not ten or more interdependent ones. SAFe organizes these teams into an Agile Release Train (ART) that plans and delivers together on a shared cadence (the Program Increment), with Program Increment planning events that surface cross-team dependencies before they become integration problems.

Roles like the Release Train Engineer and System Architect exist specifically to manage the kind of coordination and technical consistency this banking platform needs across payments, fraud detection, and credit scoring - something a collection of independent Scrum teams would struggle to achieve on their own.

6. Expected Outcomes and Limitations

Expected benefits: coordinated, predictable delivery across all teams, reduced integration risk, and a platform that meets regulatory and security requirements consistently.

How it addresses the challenges: the shared PI planning cadence surfaces cross-team dependencies early, and the ROAM risk-management approach gives every risk (dependency, regulatory, security) a clear owner and mitigation plan.

Risks and limitations: PI planning events carry real coordination overhead, and adopting SAFe requires significant organizational change and training across all participating teams.

Future enhancements: extending the ART structure to additional business lines, and introducing AI-driven risk analytics into the fraud detection engine.

Case Study 5: AI-Based Hiring System

1. Understanding the Problem Statement

A company's AI hiring system has been found to produce biased results against certain applicant groups. The problem is both an ethical one - unfair treatment of applicants - and a technical/process one, since the system needs to be redesigned to be fair, transparent, and privacy-respecting while still being useful for screening large volumes of applications.

Objectives: rebuild the hiring system so it screens candidates efficiently while eliminating unfair bias, and gives applicants transparency into how automated decisions are made.

Scope: covers resume screening, the candidate-scoring model, applicant data handling, and the human review process around automated decisions.

2. Key Stakeholders and Challenges

Job applicants: especially those from groups affected by the bias; they need fair, transparent treatment.

HR/recruiters: use the system's output to shortlist candidates and are accountable for final hiring decisions.

Data science/ML team: build and maintain the scoring model, including fixing the bias.

Legal/compliance team: ensure the system meets employment and data-protection law.

Company leadership: accountable for the reputational and legal risk of a biased hiring process.

The central challenge is balancing the efficiency benefits of automation against the real risk of the model learning and amplifying bias present in historical hiring data - and doing so without simply removing automation altogether, which would undo the practical benefit it was meant to provide.

3. Analysis of the Current Approach

Existing process: a fully automated scoring model trained on historical hiring data, with minimal human oversight of its decisions.

Strengths: fast, consistent screening that reduces manual recruiter workload at scale.

Limitations: the model reproduces bias present in the historical data it was trained on, even without explicitly using protected attributes, because other features can act as proxies for them; there is also little transparency into why a candidate was scored the way they were.

Gaps: no fairness-auditing process, no explainability for automated decisions, and no meaningful human review before an applicant is rejected.

4. Proposed Software Solution

- Data Ingestion & Bias-Auditing Module - checks training data for representation imbalance before it's used.
- Fairness-Aware Scoring Model - applies bias-mitigation techniques during scoring.
- Explainability Module - generates reason codes for each candidate's score.
- Human Review Interface - lets recruiters review and override automated decisions, especially rejections.
- Applicant Transparency Portal - informs applicants that automation is involved and shows key decision factors.

Functional requirements: resume parsing, candidate scoring, explanation generation, human override capability, and applicant consent capture.

Non-functional requirements: fairness (bias metrics monitored continuously, not just at launch), privacy (data minimization and encryption of applicant PII), transparency, and usability for recruiters reviewing flagged cases.

Software engineering principles applied: modularity (the scoring engine is decoupled from the decision workflow, so it can be updated or replaced as fairness techniques improve without touching the rest of the system); scalability (handles high applicant volumes); reliability (consistent, reproducible scoring); usability (clear recruiter dashboard and applicant-facing transparency); and security (encrypted storage and strict access control over sensitive applicant data) - extended here with an explicit ethical-design layer of fairness audits and explainability that a standard hiring tool would not normally need.

5. Justification of Chosen Methodology/Framework

An Agile approach with dedicated ethics/fairness review checkpoints built into each iteration is the most suitable choice here, rather than a one-time Waterfall-style build-and-ship. Fairness issues in AI systems are rarely fixed permanently in a single pass - they need to be re-evaluated as the model is retrained on new data.

Agile's iterative cadence allows a fairness audit and explainability check to be treated as a recurring part of every release cycle, rather than a box ticked once before launch. This matches the ongoing, evolving nature of algorithmic fairness far better than a linear model that treats ethical review as a single upfront gate.

6. Expected Outcomes and Limitations

Expected benefits: fairer and more transparent hiring decisions, reduced legal and reputational risk, and improved applicant trust in the process.

How it addresses the challenges: continuous bias auditing and human review directly target the root cause of the original problem, while explainability restores some of the transparency a black-box model removed.

Risks and limitations: bias may persist in subtler forms even after mitigation, different fairness metrics can conflict with each other so perfect fairness across all of them is not always achievable, and ongoing monitoring requires sustained resourcing rather than a one-time fix.

Future enhancements: continuous fairness-monitoring dashboards, independent third-party bias certification, and broader sourcing integrations aimed at improving applicant diversity earlier in the pipeline.